

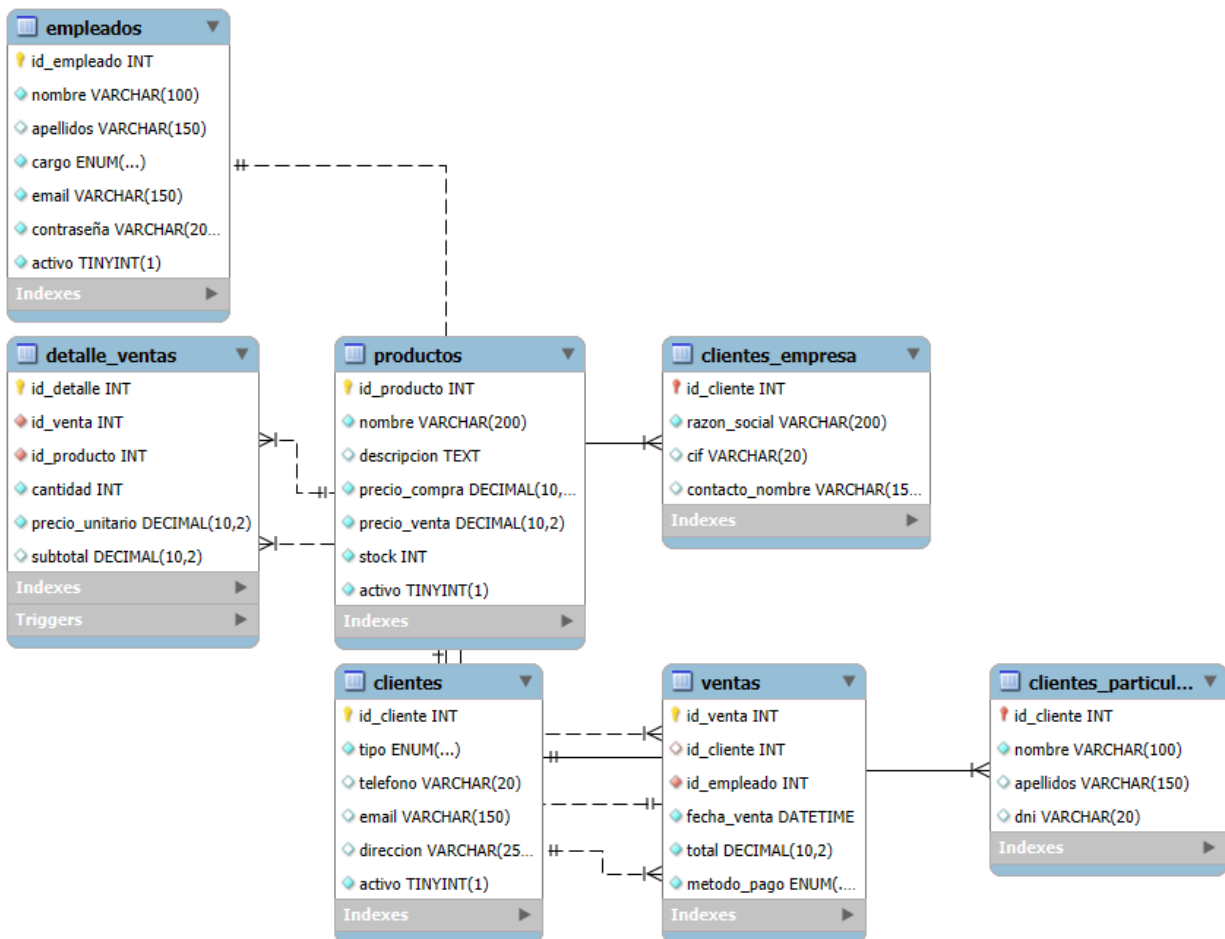
Fase 1: Exploración e Investigación

1.1. Análisis del Dominio del Problema

Para el desarrollo de nuestro sistema de gestión para la tienda de informática, la primera etapa consistió en comprender las necesidades reales de un negocio de este sector. A través del análisis del problema, determinamos que el sistema requería integrar tres grandes bloques funcionales:

1. **Gestión Comercial:** Clientes, ventas, facturación y métodos de pago.
2. **Gestión de Almacén:** Control de stock, inventario de productos, alertas de stock bajo y relación con proveedores.
3. **Servicio Técnico:** Registro de reparaciones, asignación de piezas, control de estados (pendiente, en proceso, finalizado) y asignación de técnicos.

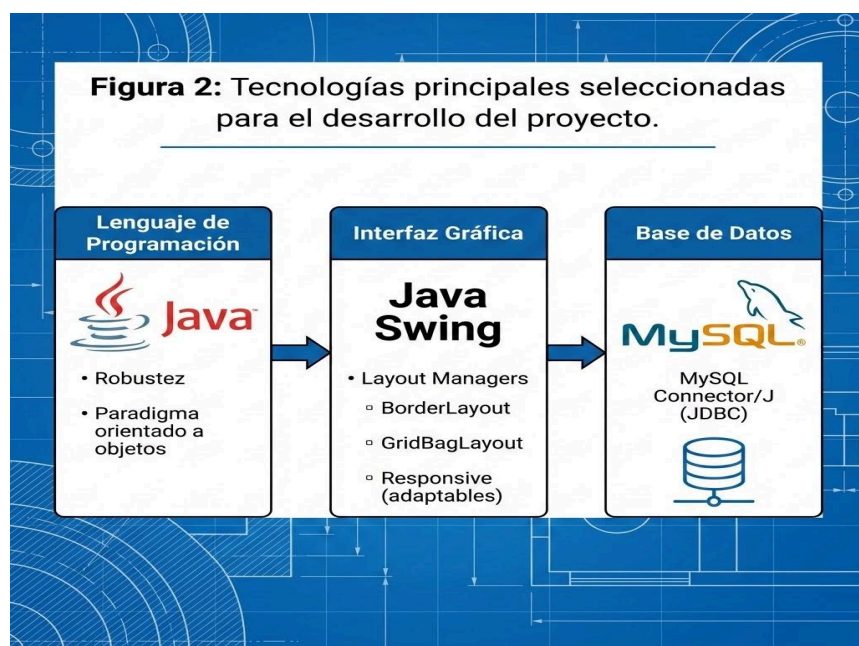
Esta complejidad requirió un diseño robusto de la base de datos (Modelo Entidad-Relación) y una arquitectura de software en Java capaz de manejar distintos roles de usuario (Administrador, Técnico, Vendedor).



1.2. Tecnologías y Arquitectura Seleccionadas

Durante la fase de exploración técnica, evaluamos qué pila tecnológica (Stack) sería la más adecuada para cumplir con los requisitos multiplataforma y de seguridad del proyecto:

- **Lenguaje de Programación:** Java (por su robustez y paradigma orientado a objetos).
- **Interfaz Gráfica:** Java Swing, investigando específicamente el uso de *Layout Managers* (como BorderLayout y GridBagLayout) para asegurar que las ventanas fueran *responsive* (adaptables a distintas resoluciones de pantalla).
- **Base de Datos:** MySQL, utilizando la librería *MySQL Connector/J* (JDBC) para establecer la conexión desde Java.

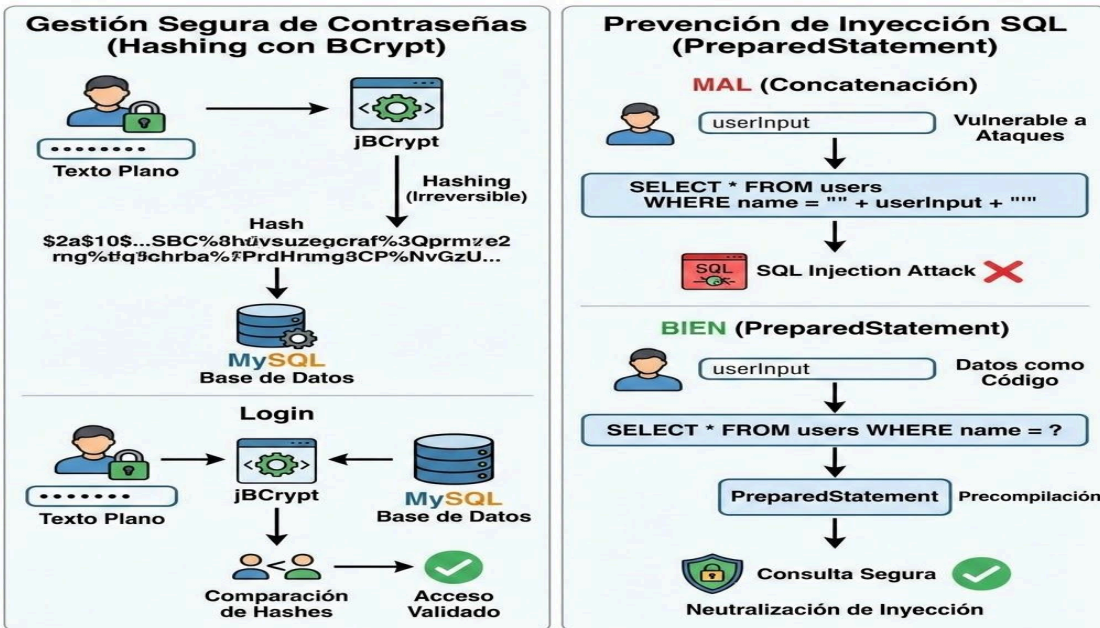


1.2.1. Enfoque en la Seguridad de la Aplicación

Para garantizar la integridad del sistema y la privacidad de los usuarios, se han investigado e implementado dos estándares de seguridad críticos:

- **Gestión Segura de Contraseñas (Hashing con BCrypt):** Se descartó desde un principio almacenar las contraseñas en texto plano. En su lugar, se seleccionó la librería externa *jBCrypt*. Cuando un usuario (empleado) se registra, la aplicación aplica un proceso de *hashing* matemático irreversible a su contraseña antes de enviarla a MySQL. Durante el *login*, el sistema no "desencripta" la contraseña, sino que utiliza los métodos de BCrypt para comparar el texto introducido con el *hash* almacenado en la base de datos, validando el acceso de forma 100% segura.
- **Prevención de Inyección SQL (PreparedStatement):** En lugar de concatenar cadenas de texto (String) para armar las consultas SQL de inserción o búsqueda, la conexión JDBC se

configuró para usar objetos de tipo PreparedStatement. Esta técnica "precompila" la consulta en la base de datos y trata las variables introducidas por los usuarios estrictamente como datos, neutralizando cualquier intento de ataque por inyección de código SQL malicioso.



1.3. Fuentes de Información y Herramientas de Apoyo

Para resolver los desafíos técnicos que surgieron durante el diseño de la base de datos y la codificación en Java, el equipo recurrió a diversas fuentes de conocimiento y herramientas de asistencia:

A. Foros y Comunidades de Desarrollo

- **StackOverflow:** Ha sido una fuente fundamental para la resolución de errores específicos (*debugging*). Se consultaron hilos y discusiones previas para resolver problemas concretos, como excepciones de conexión de JDBC (`ClassNotFoundException`), la correcta sintaxis para los PreparedStatement de seguridad, y técnicas para hacer las ventanas de Swing adaptables.
- **GitHub:** Se exploraron repositorios públicos con proyectos similares (Puntos de Venta o ERPs básicos en Java) para analizar arquitecturas de código y entender cómo estructurar el proyecto en paquetes lógicos (Modelo-Vista-Controlador).

B. Inteligencia Artificial como Asistente de Desarrollo

En lugar de buscar únicamente en la documentación tradicional, integramos herramientas de Inteligencia Artificial Generativa como apoyo al aprendizaje y la aceleración del desarrollo:

- **Google Gemini:** Utilizado como asistente de arquitectura y seguridad. Gemini nos ayudó a generar y estructurar los diagramas UML (Casos de Uso, Clases) adaptados a la sintaxis de *draw.io* y *Mermaid*, a comprender a nivel profundo el flujo de trabajo de la encriptación y validación de contraseñas con BCrypt, y a optimizar la estructura de nuestros scripts SQL.
- **Anthropic Claude:** Empleado como herramienta de revisión. Se utilizó para analizar nuestro diseño del Modelo Entidad-Relación (E-R), comprobar que las cardinalidades eran correctas y recibir sugerencias sobre la normalización de tablas intermedias (como la relación entre Reparaciones y Productos a través de la tabla de Piezas).

C. Documentación Oficial

- **Oracle Java Documentation:** Para consultar los métodos exactos de las clases de `java.sql` (`Connection`, `ResultSet`, `PreparedStatement`) y los componentes de Swing.
- **MySQL Reference Manual:** Para la correcta definición de los tipos de datos en la creación de las tablas (ej. diferencias entre `VARCHAR` y `TEXT`, o el uso de `ENUM` para los estados y cargos).

D. Videos

- **BCrypt:**  [Java Bcrypt example - salt + pepper](#)

1.4. Organización

Para hacer esta parte del trabajo, nos hemos organizado repartiendo lo que tenemos que hacer por asignaturas. Nos hemos ayudado de la inteligencia artificial (Claude) para mejorar la información que ya teníamos y que el proyecto quede más completo y mejor. Además, hemos usado Gemini para generar imágenes que vamos a poner en la página web y para buscar datos concretos que nos hacían falta.

Aquí os contamos lo que estamos usando en cada asignatura:

- **Programación:** Usamos **NetBeans** porque es el que hemos usado todo el año en clase y es con el que nos sentimos más cómodos. Ya sabemos cómo empezar a programar y hemos aprendido cómo enlazar el programa con la base de datos.
- **Entornos de Desarrollo:** Para hacer los diagramas (de comportamiento, clases y el Modelo Entidad-Relación) hemos usado **Draw.io**. Lo elegimos porque es muy fácil de usar, los diagramas quedan muy claros y visuales, y además es el que nos recomendaron los profesores. Ya tenemos los diagramas hechos.
- **Bases de Datos:** Estamos usando **MySQL Workbench** porque es lo mismo, nos sentimos cómodos porque es el que hemos usado siempre en clase. Ya tenemos la base de datos montada con las herencias.
- **Lenguaje de Marcas:** Para la web, nos hemos basado en proyectos de otros años que hicimos con Alberto. También nos ha ayudado Claude para sacar ideas nuevas y

Gemini para buscar información y crear imágenes para la página. Usamos **Visual Studio Code** porque es el que hemos usado durante todo el curso. Ahora mismo estamos empezando a montar la página.

- **Sistemas Informáticos:** En esta fase lo que hemos hecho ha sido emplear una máquina virtual de Debian donde hemos configurado un servidor apache, donde hemos añadido un certificado SSL, y una sección de la página que es privada y solo se puede acceder con nuestro usuario y contraseña, para pasar los archivos de la web a Debian hemos empleado la aplicación FileZilla, donde hemos arrastrado los archivos que componen la página y de esta forma podemos ver la Web tanto en una máquina de Windows normal como en la máquina virtual de Debian.

1.5. Conclusión de la Fase

Gracias a esta investigación previa, el equipo ha logrado definir un Modelo Entidad-Relación sólido que soporta todas las operaciones de la tienda, y ha establecido unas bases claras sobre cómo implementar una conexión segura entre Java y MySQL. La incorporación de BCrypt y PreparedStatement asegura un entorno confiable y minimiza los riesgos. Además, la utilización estratégica de inteligencias artificiales como Claude y Gemini como asistentes de desarrollo ha sido vital para agilizar la resolución de dudas técnicas, optimizar el diseño de la base de datos y prepararnos sólidamente de cara a la fase de Desarrollo.